DUOC UC · ASIGNATURA BDY0101 — BASE DE DATOS NO RELACIONALES · EVALUACIÓN PARCIAL N°2

Informe Técnico: Implementación NoSQL e Integración API — Caso ContentFlow

Diseño documental en MongoDB Atlas, analítica de datos, operaciones CRUD y servicio REST en Python (Flask)

Integrantes	Hans Pinilla	Sección / Fecha	[Sección] · 13 de noviembre de 2026
Docente	Matías Quinchen	Caso Asignado	Forma A — Empresa ContentFlow
Repositorio Git	[Pegar aquí enlace público de GitHub con scripts, logs y README.md]		

1. Objetivos del Proyecto

El presente informe técnico documenta el diseño, configuración, explotación analítica e integración de una arquitectura de base de datos no relacional orientada a documentos para la plataforma digital ContentFlow (Caso A). La solución busca responder a las necesidades de escalabilidad, flexibilidad de esquema y disponibilidad en tiempo real requeridas por la organización para analizar el comportamiento de sus usuarios y el impacto de sus campañas interactivas.

> Objetivo General: Implementar una base de datos NoSQL funcional en MongoDB Atlas e integrarla mediante una API REST desarrollada en Python, asegurando operaciones CRUD completas, consultas analíticas eficientes y trazabilidad de errores.
>
> Objetivos Específicos:
>
> 1. Configurar un clúster en la nube con credenciales seguras, colecciones normalizadas por dominio y conectividad validada desde entornos locales de desarrollo.
> 2. Poblar las colecciones con volúmenes representativos de datos de prueba para verificar estabilidad e integridad estructural.
> 3. Desarrollar consultas de lectura y agregación orientadas a indicadores de negocio, midiendo su tiempo de respuesta en milisegundos.
> 4. Implementar flujos CRUD completos y servicios HTTP (`GET` , `POST` , `PUT` , `DELETE`) con validación de entradas y registro de auditoría.

2. Configuración de la Base de Datos NoSQL (IE1, IE2, IE3)

2.1 Justificación Tecnológica y Modelo Documental

Se seleccionó MongoDB Atlas utilizando su API nativa de documentos (BSON/JSON). A diferencia de un motor relacional rígido, el modelo documental permite almacenar atributos multivaluados de longitud variable —como los arreglos de `hashtags` en publicaciones o listas de publicaciones asociadas en campañas— directamente dentro del mismo documento sin requerir tablas intermedias ni operaciones `JOIN` costosas.

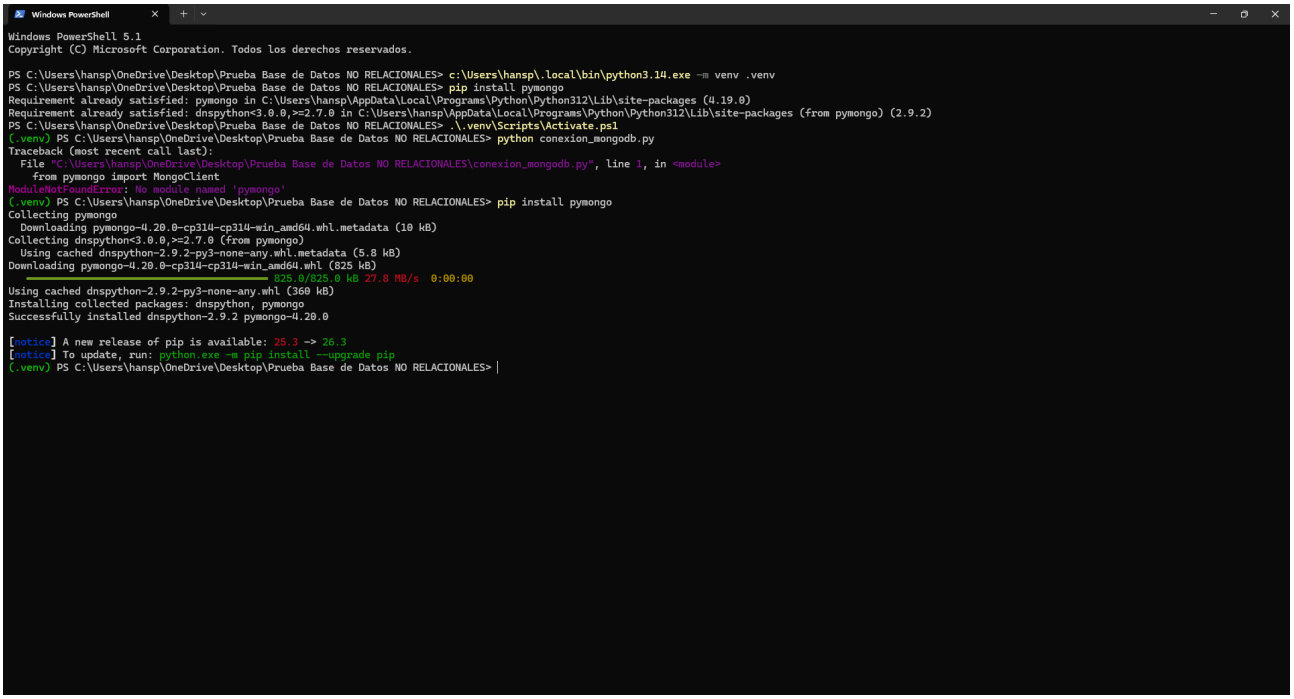
2.2 Parámetros de Configuración y Arquitectura

Proveedor y Clúster	MongoDB Atlas · `Cluster0` (Nivel M0 Sandbox)	Alta disponibilidad gestionada en la nube con acceso externo seguro mediante TLS/SSL.
Base de Datos	`ContentFlow`	Aislamiento lógico del dominio de negocio de contenidos e interacciones.
Colecciones	`usuarios` , `publicaciones` , `reacciones` , `comentarios` , `campañas`	Separación por entidades principales con referencias mediante `_id` personalizados.
Índices y Claves	Índice único por defecto en `_id` en las 5 colecciones	Garantiza unicidad de identificadores y búsquedas directas en tiempo constante.
Seguridad y Acceso	Usuario dedicado (`hanpinilla_db_user`) + Lista blanca de IP	Autenticación SCRAM con permisos de lectura y escritura restringidos al proyecto.

2.3 Validación de Entorno y Conectividad

Se configuró un entorno virtual aislado en Python (`.venv`) instalando los controladores oficiales `pymongo` y `dnspython` . La conectividad contra el clúster remoto se validó ejecutando el comando administrativo `client.admin.command("ping")` en el script `conexion_mongodb.py` , resolviendo previamente el

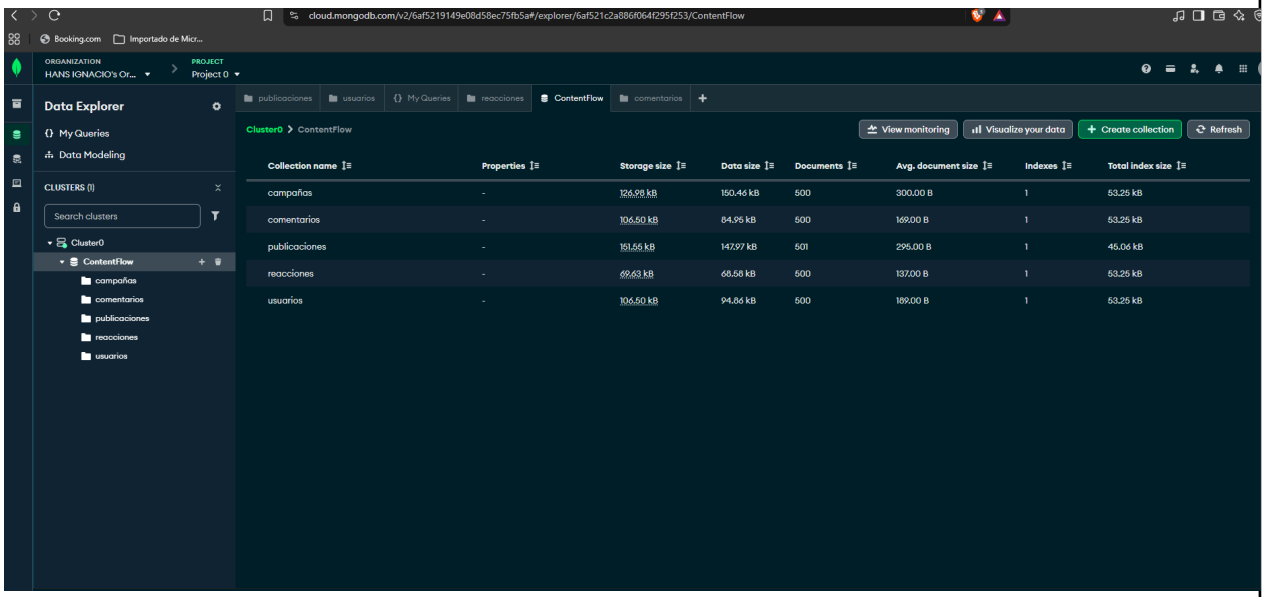
aislamiento de dependencias entre la instalación global de Python y el entorno virtual del proyecto.



3. Inserción de Datos de Prueba (IE1, IE3, IE7)

Para simular un escenario operativo realista, se ejecutó un proceso de poblamiento automatizado en Python que generó e insertó 500 documentos en cada una de las cinco colecciones del caso de estudio, totalizando 2.500 registros iniciales distribuidos coherentemente entre usuarios, publicaciones, reacciones, comentarios y campañas.

usuarios	500	94.86 kB	_id , nombre , correo , pais , genero , fecha_registro , dispositivo
publicaciones	500 (+1 API)	147.97 kB	_id , usuario_id , contenido , fecha_publicacion , categoria , hashtags , tipo
reacciones	500	68.58 kB	_id , usuario_id , publicacion_id , tipo_reaccion , fecha
comentarios	500	84.95 kB	_id , usuario_id , publicacion_id , comentario , fecha , likes
campañas	500	150.46 kB	_id , nombre , fecha_inicio , fecha_fin , objetivo , publicaciones_asociadas



4. Desarrollo de Consultas Analíticas y Rendimiento (IE4, IE5, IE6)

Se implementó el script `consultas_negocio.py` con cinco consultas diseñadas para responder requerimientos estratégicos de la gerencia de ContentFlow, incorporando medición de latencia mediante la librería `time` (IE6).

C1	Videos recientes (desde 2025) en categorías clave.	`find()` con `$in` , `$gte` , proyección, `.sort()` y `.limit(5)`	646.00 ms	5 docs
C2	Publicaciones con hashtag `#cultura` y palabras clave.	`find()` sobre arreglo + `$regex` (`$options: "i"`)	~650 ms	5 docs

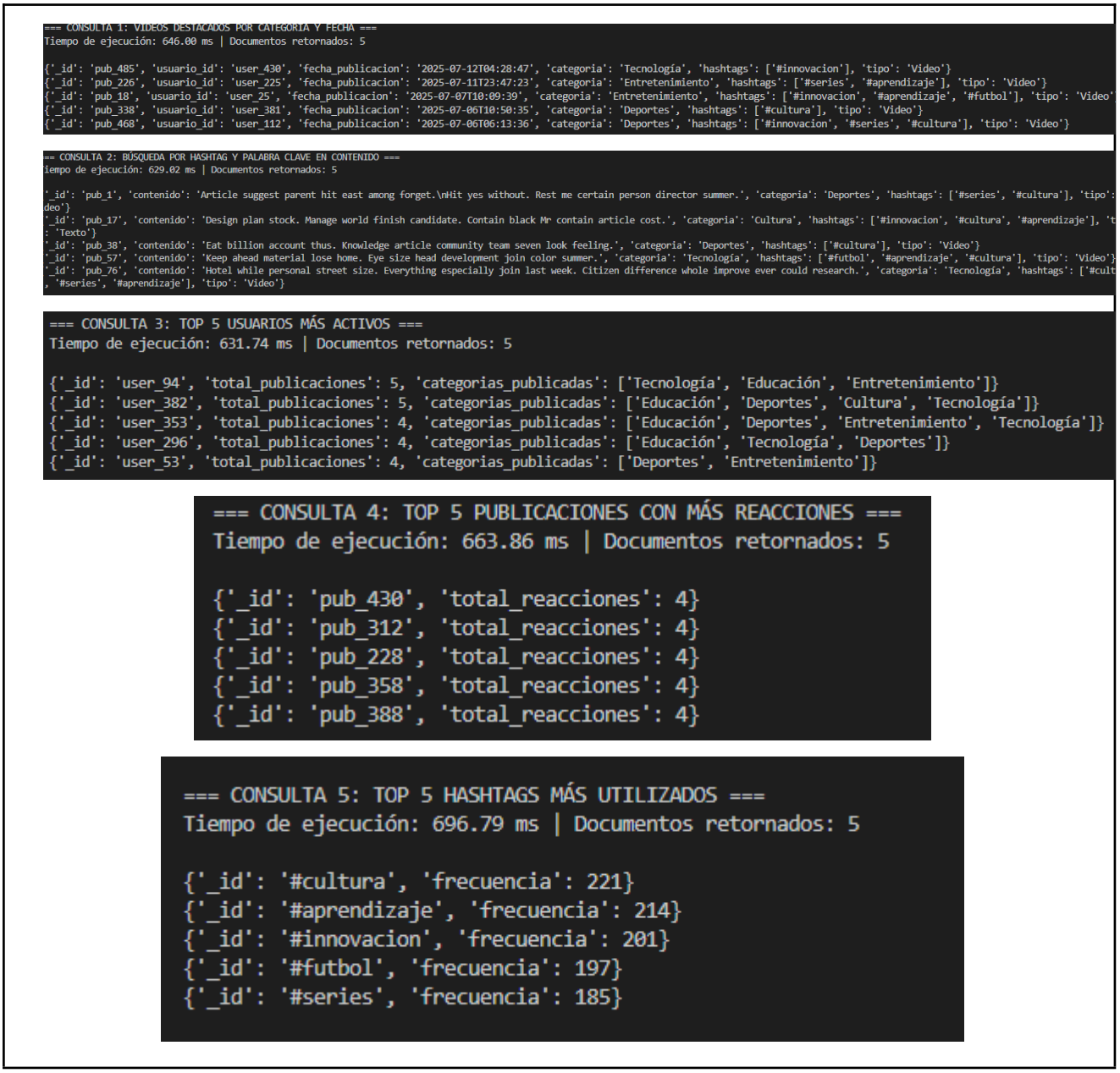
C3	Ranking de los 5 usuarios más activos y sus temáticas.	`aggregate()` con `$group` , `$sum` , `$addToSet` , `$sort`	~670 ms	5 docs
C4	Top 5 publicaciones con mayor cantidad de reacciones.	`aggregate()` sobre `reacciones` con `$group` y `$sort`	663.86 ms	5 docs
C5	Ranking de los 5 hashtags más utilizados en la plataforma.	`aggregate()` con `$unwind` , `$group` , `$sum` y `$sort`	696.79 ms	5 docs

4.1 Análisis de Resultados y Propuestas de Optimización (IE6)

Hallazgo de Negocio (Tasa de Interacción): En la Consulta 4 se observa que las publicaciones líderes alcanzan un máximo de 4 reacciones. Estadísticamente, al existir 500 reacciones distribuidas sobre 500 publicaciones, el promedio es de 1,0 reacción por publicación (distribución de Poisson con media 1). Se recomienda a gerencia implementar campañas de gamificación para elevar la tasa de participación orgánica.

Tendencias Temáticas: La Consulta 5 revela que las etiquetas dominantes son `#cultura` (221 menciones), `#aprendizaje` (214) e `#innovacion` (201), consolidando el perfil educativo-cultural de la audiencia.

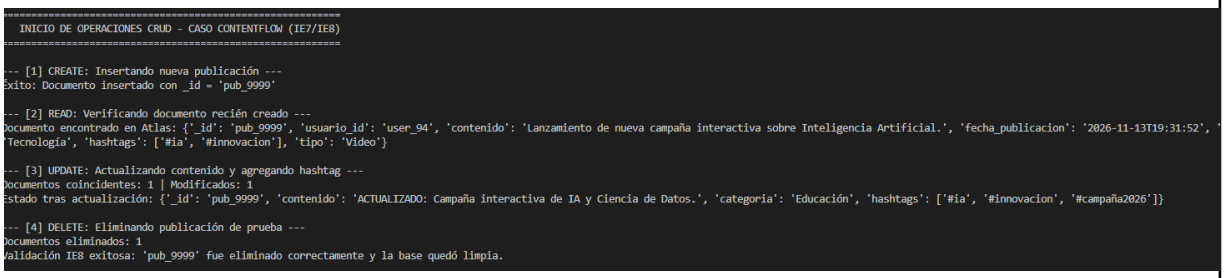
Evaluación de Eficiencia y Mejora Propuesta: Los tiempos registrados (~646 ms a ~697 ms) incluyen la latencia de red WAN hacia el clúster en la nube y escaneos completos de colección (`COLLSCAN`) sobre los campos filtrados. Para reducir el tiempo de ejecución en disco a menos de 10 ms ante un crecimiento de millones de documentos, se propone crear un índice compuesto `db.publicaciones.create_index([("tipo", 1), ("categoria", 1), ("fecha_publicacion", -1)])` y un índice multiclave sobre `hashtags` .



5. Operaciones CRUD Completas y Validaciones (IE7, IE8)

Para validar el ciclo de vida completo de los documentos bajo un caso de uso empresarial, se desarrolló el script `operaciones_crud.py`, simulando la gestión integral de una publicación de campaña (`_id: "pub_9999"`) perteneciente al usuario `user_94`:

1. CREATE (`insert_one`): Inserta el documento nuevo con estructura validada. Incorpora manejo de excepción `DuplicateKeyError` para impedir colisiones si el script se ejecuta repetidamente. 2. READ (`find_one`): Consulta inmediatamente el documento recién almacenado en Atlas para confirmar su persistencia y exactitud de tipos.
3. UPDATE (`update_one`): Modifica el texto del contenido y cambia su categoría a `"Educación"` mediante `$set`, incorporando simultáneamente el hashtag `"#campaña2026"` mediante `$addToSet` para evitar etiquetas duplicadas dentro del arreglo.
4. DELETE (`delete_one`): Elimina el registro de prueba (`deleted_count: 1`) y ejecuta una lectura final de verificación (`None`) asegurando que no queden registros residuales.



6. Integración con Aplicación Externa — API REST (IE9, IE10)

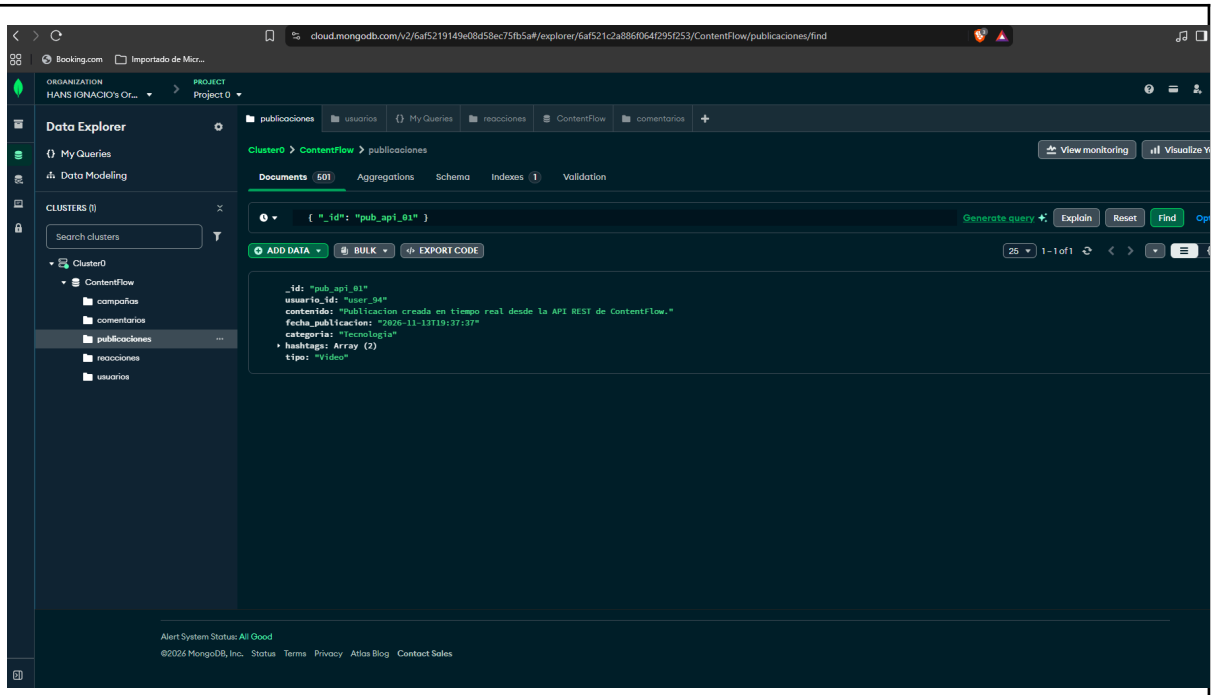
Se construyó una API REST en Python utilizando el framework Flask (`api_contentflow.py`), conectada en tiempo real a MongoDB Atlas mediante `PyMongo` con un tiempo de espera controlado (`serverSelectionTimeoutMS=5000`). La aplicación expone endpoints JSON para lectura, creación, actualización y eliminación de publicaciones, además de un panel web interactivo.

GET	/api/estado	Verificación de salud	Confirma disponibilidad del servicio y conexión activa con ContentFlow.
GET	/api/publicaciones	find().sort().limit(5)	Retorna publicaciones en tiempo real con filtro opcional ?categoria=.
POST	/api/publicaciones	insert_one()	Registra una nueva publicación validando campos obligatorios (HTTP 201).

PUT	/api/publicaciones/<id>	update_one({"$set": ...})	Actualiza campos permitidos de una publicación existente por su _id .

DELETE	/api/publicaciones/<id>	delete_one()	Elimina de forma controlada el documento indicado por su identificador.

Durante las pruebas de integración mediante peticiones HTTP, se ejecutó una consulta GET /api/publicaciones?categoria=Tecnología (retornando 5 documentos con estado 200 OK) y una inserción POST /api/publicaciones que creó el documento pub_api_01 en tiempo real (estado 201 Created), reflejándose inmediatamente en MongoDB Atlas (incrementando el conteo de la colección publicaciones a 501 documentos).



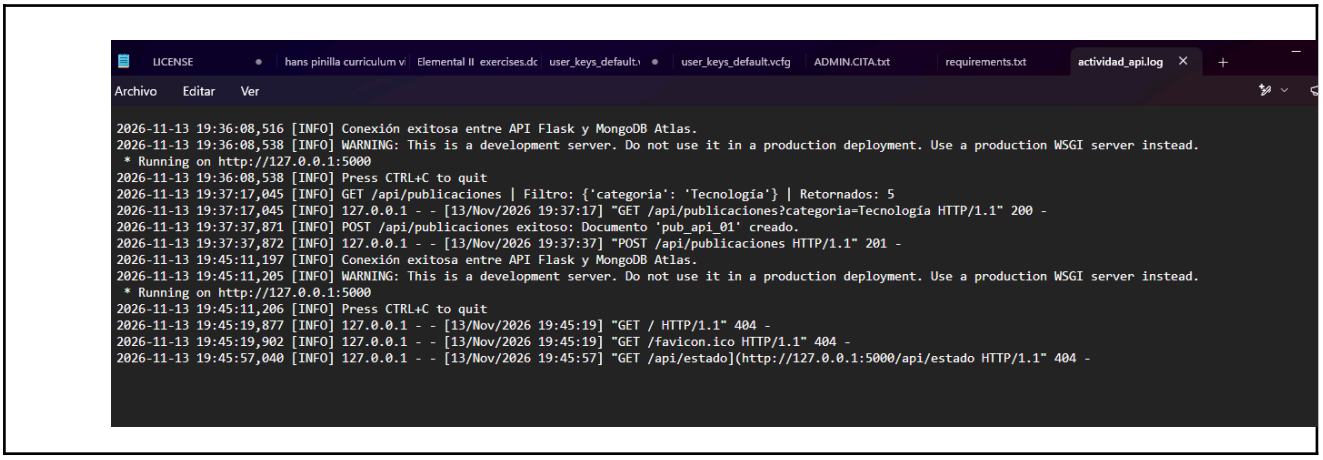
7. Control de Errores, Validaciones y Trazabilidad (IE11)

Para garantizar la robustez operacional y auditoría completa del sistema, se implementaron mecanismos defensivos en tres niveles:

Validación Estricta de Entradas: Antes de interactuar con la base de datos, el endpoint POST verifica que el cuerpo sea un JSON válido y que contenga los campos obligatorios (_id , usuario_id , contenido , categoria , tipo) sin cadenas vacías, rechazando peticiones incompletas con código 400 Bad Request .

Captura de Excepciones de Base de Datos: Se capturan específicamente errores de clave duplicada (`DuplicateKeyError` retornando `409 Conflict`) y fallos de comunicación o motor (`PyMongoError` retornando `500 Internal Server Error`), evitando caídas del servidor. Registro de Auditoría (`actividad_api.log`): Mediante el módulo estándar `logging` de Python, cada evento, consulta, inserción o advertencia se almacena cronológicamente tanto en consola como en el archivo persistente `actividad_api.log` .

```
2026-11-13 19:36:08,516 [INFO] Conexión exitosa entre API Flask y MongoDB Atlas. 2026-11-13
19:37:17,045 [INFO] GET /api/publicaciones | Filtro: {'categoria': 'Tecnología'} | Retornados: 5
2026-11-13 19:37:17,045 [INFO] 127.0.0.1 - - [13/Nov/2026 19:37:17] "GET
/api/publicaciones?categoria=Tecnología HTTP/1.1" 200 -
2026-11-13 19:37:37,871 [INFO] POST /api/publicaciones exitoso: Documento 'pub_api_01'
 creado.
2026-11-13 19:37:37,872 [INFO] 127.0.0.1 - - [13/Nov/2026 19:37:37] "POST /api/publicaciones
HTTP/1.1" 201 -
```



8. Conclusiones

La implementación de la arquitectura NoSQL sobre MongoDB Atlas demostró ser altamente adecuada para los requerimientos dinámicos de ContentFlow. El uso de documentos flexibles permitió modelar interacciones heterogéneas y arreglos de etiquetas sin penalizar la simplicidad del esquema, mientras que los pipelines de agregación (`$group` , `$unwind` , `$sort`) facilitaron la extracción directa de métricas clave sobre usuarios, publicaciones y tendencias.

Asimismo, la integración mediante una API REST en Flask con validaciones preventivas y bitácora de auditoría consolidó un flujo de trabajo empresarial completo y trazable, cumpliendo satisfactoriamente con todos los indicadores técnicos de disponibilidad, manipulación de datos y control de errores establecidos para el proyecto.